Projecto Github

Integrantes: Miguel Angel Colobon Martinez
ISAAC ARAY
Miiguel Angel Godoy Gamboa

Instructor: David Cabezas
Fecha: 20/04/2026

¿Que es GitHub?

GitHub es una plataforma en la nube que utiliza un sistema llamado **Git**. Por ejemplo imaginalo como una red social para programadores y una biblioteca gigante de código.

- **El Motor (Git):** Es una herramienta que instalas en tu computadora para rastrear los cambios en tus archivos. Es como el "Historial de versiones" de un documento, pero para miles de archivos a la vez.
- **La Nube (GitHub):** Es el lugar donde subes esos archivos para que estén seguros, para compartirlos con otros o para trabajar en equipo.

¿Que es un commit?

Un **commit** es una "captura de pantalla" o un punto de guardado de tu proyecto en un momento específico.

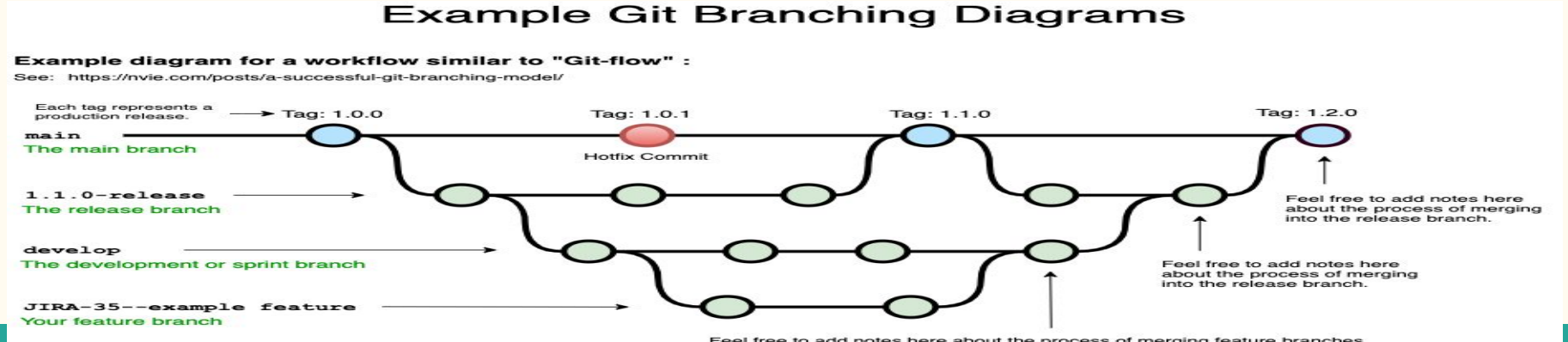
No es simplemente guardar un archivo (como hacer `Ctrl + S`). Cuando haces un commit, le dices al sistema: *"Registra exactamente cómo se ve todo mi proyecto en este instante y asígnale un mensaje explicativo"*.

¿Por qué es importante? Si mañana borras algo por error o tu código deja de funcionar, puedes regresar exactamente al estado de un commit anterior. Cada commit tiene un identificador único y un autor.

¿Qué es branch?

En la página de un repositorio, las ramas son fundamentales para la colaboración.

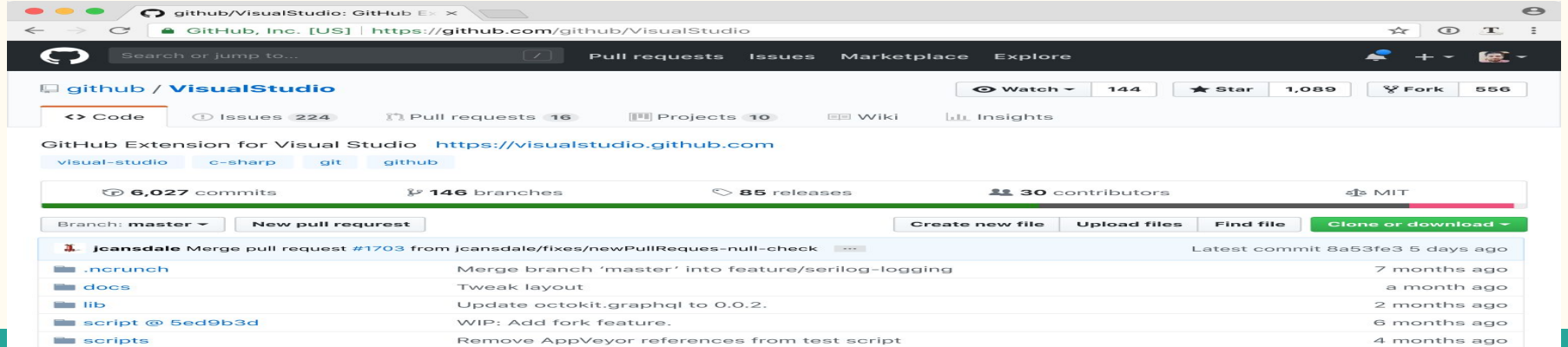
- **El selector de ramas:** Arriba de la lista de archivos, verás un botón desplegable que suele decir "**main**" o "**master**". Si haces clic en él, verás todas las ramas que existen.
- **Crear una rama:** Desde ese mismo botón puedes escribir un nombre nuevo y crear una rama ahí mismo. Esto "copia" el estado actual del código para que puedas editarlo sin miedo.
- **El Pull Request (La magia de GitHub):** Cuando trabajas en una rama (por ejemplo, una llamada `mejorar-diseño`) y subes tus cambios a GitHub, la página te invitará a crear un **Pull Request**.
  - Un *Pull Request* es básicamente decir: *"Oye, tengo estos cambios en mi rama, ¿puedes revisarlos y fusionarlos (Merge) a la rama principal?"*.



¿Qué es repossitory?

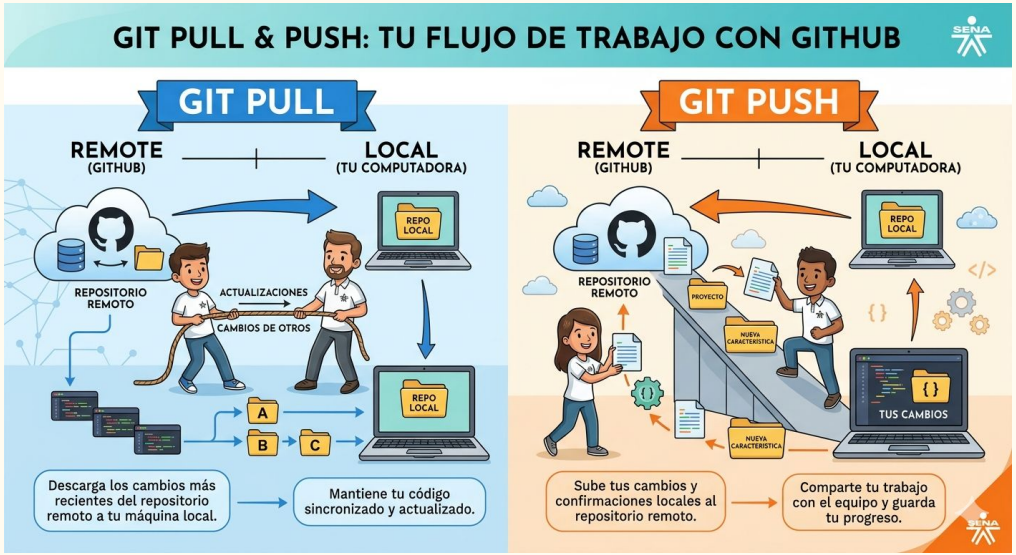
Cuando entras a un proyecto en GitHub (por ejemplo, `github.com/usuario/proyecto`), lo que ves es el repositorio.

- **La "Home" del proyecto:** Todo lo que ves en la pestaña principal (archivos, carpetas, el archivo README) es el contenido del repositorio.
- **El historial (Commits):** En la parte superior derecha verás un icono de un reloj con un número de "**Commits**". Si haces clic ahí, verás toda la "línea de tiempo" de quién cambió qué y cuándo. Es como el registro de cambios de un documento compartido.
- **Visibilidad:** En GitHub, el repositorio puede ser **Público** (cualquiera en internet puede verlo y aprender de tu código) o **Privado** (solo tú y las personas que invites pueden verlo).



PUSH AND PULL (GIT HUB)

En GitHub Desktop, los comandos *push* y *pull* son fundamentales para trabajar en equipo, ya que permiten enviar y recibir cambios entre nuestro computador y el repositorio en la nube.



⏫ 2. ¿Qué es PUSH?

El comando push se utiliza para subir los cambios locales al repositorio remoto, permitiendo que otros miembros del equipo vean las actualizaciones.”

📌 Ejemplo:

- Editas un archivo
- Guardas cambios
- Haces commit
- Presionas **Push origin** en GitHub Desktop

🔽 3. ¿Qué es PULL?

El comando pull permite descargar los cambios más recientes del repositorio remoto para mantener nuestro proyecto actualizado.

📌 Ejemplo:

- Un compañero sube cambios
- Tú haces clic en **Pull origin**
- Tu proyecto se actualiza

🔄 4. Relación entre Push y Pull

Push y pull trabajan juntos: primero traemos cambios con pull para evitar conflictos, y luego enviamos nuestros cambios con push

Importancia

.Permiten trabajo en equipo

.Evitan perder información

.Mantienen el proyecto sincronizado

GitHub Desktop

Es una aplicación gráfica (no necesitas usar comandos)

Tiene botones como:

.Fetch origin

.Pull origin

.Push origin

CONCLUSIÓN

.push y pull son esenciales

. actualizado un proyecto

.Facilitar el trabajo colaborativo

NOTA:Siempre hacer **pull antes de push**

Evita conflictos en el código

Trabajar en equipo se vuelve más organizado

Como compartir un repositorio

Para compartir un repositorio existen 2 maneras dependiendo de como quieren que colaboren 1. para que solo vea el codigo y 2. para que colabore activamente editandolo.

1. Compartir mediante la URL (Lectura)

Si tu repositorio es **Público**, cualquier persona con el enlace puede ver los archivos, descargarlos o clonarlos en su propia computadora.

- Entra a tu repositorio en GitHub.
- Copia la URL de la barra de direcciones de tu navegador (ej: `https://github.com/tu-usuario/tu-repositorio`).
- Envía ese enlace a quien quieras.

> **Nota:** Si el repositorio es **Privado**, el enlace solo te funcionará a ti. Para que otros lo vean, debes seguir el paso 2.

2. Invitar Colaboradores (Edición)

Si quieres que un amigo o colega pueda hacer `push` (subir cambios) directamente a tu repositorio, debes darle permisos de acceso:

1. En la página de tu repositorio en GitHub, haz clic en la pestaña **Settings** (Configuración).
2. En el menú de la izquierda, selecciona **Collaborators**.
3. Haz clic en el botón verde **Add people**.
4. Busca a la persona por su **nombre de usuario** o por su **correo electrónico**.
5. GitHub le enviará una invitación por correo. **Esa persona debe aceptar la invitación** para poder ver o editar el repositorio (especialmente si es privado).

¿Qué es Github Desktop?

En términos sencillos, **GitHub Desktop** es una aplicación visual que instalas en tu computadora para gestionar tus proyectos de programación sin tener que escribir comandos complicados en una terminal (la famosa pantalla negra).

Es como pasar de usar una calculadora científica de texto a usar una aplicación con botones, menús y gráficas. Es la "cara amigable" de Git, el sistema que registra todos los cambios en tu código.

¿Para qué sirve realmente? (Funciones clave)

Si estás trabajando en un proyecto (como tus scripts de Python o tus aplicaciones en HTML), GitHub Desktop te ayuda a:

1. **Visualizar cambios:** Ver exactamente qué líneas de código borraste, añadiste o modificaste antes de guardar los cambios.
2. **Sincronizar con la nube:** Enviar tu código a GitHub.com con un solo clic para tener un respaldo o compartirlo.
3. **Historial visual:** Ver una línea de tiempo de todo lo que has hecho en el proyecto desde el día uno.
4. **Trabajo en equipo:** Descargar las actualizaciones que tus compañeros hayan hecho sin conflictos extraños.